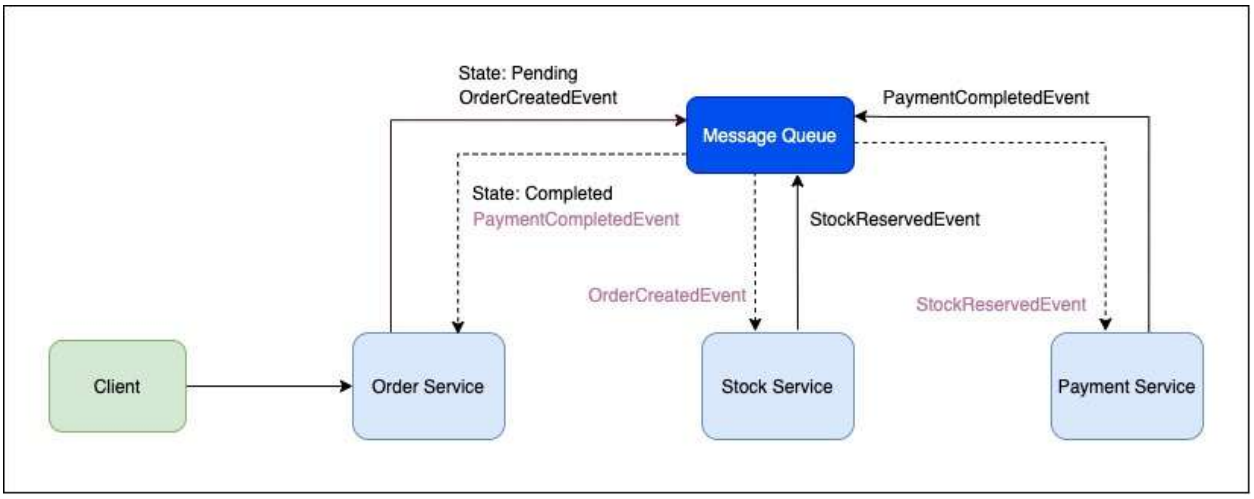
Below is a **complete, classroom-ready ASP.NET Core implementation** of the **Order–Payment–Inventory Choreography Saga** using **Azure Service Bus Topics**.
The code is **production-aligned but simplified for learning**.

---

0. Architecture Recap (What We Are Building)



Microservices

- OrderService (starts & finalizes saga)
- PaymentService (charges/refunds)
- InventoryService (reserves stock)

Messaging

- **Azure Service Bus Topic**
- Each service:
    - Publishes events
    - Subscribes to relevant events
    - Has its own database

---

1. Shared Contracts (IMPORTANT)

Create a **Shared.Contracts** class library.

Events/BaseEvent.cs

```
namespace Shared.Contracts;

public abstract class BaseEvent
{
    public Guid CorrelationId { get; set; }
    public DateTime OccurredOn { get; set; } = DateTime.UtcNow;
}
```

Events

```
namespace Shared.Contracts;

public class OrderCreatedEvent : BaseEvent
{
    public int OrderId { get; set; }
    public decimal Amount { get; set; }
}

public class PaymentCompletedEvent : BaseEvent
{
    public int OrderId { get; set; }
}

public class PaymentFailedEvent : BaseEvent
{
    public int OrderId { get; set; }
    public string Reason { get; set; }
```

```
}

public class InventoryReservedEvent : BaseEvent
{
    public int OrderId { get; set; }
}

public class InventoryFailedEvent : BaseEvent
{
    public int OrderId { get; set; }
    public string Reason { get; set; }
}

public class PaymentRefundedEvent : BaseEvent
{
    public int OrderId { get; set; }
}
```

---

2. Azure Service Bus Helper (Used by All Services)

Infrastructure/ServiceBusPublisher.cs

```
using Azure.Messaging.ServiceBus;
using System.Text.Json;

public class ServiceBusPublisher
{
    private readonly ServiceBusClient _client;
```

```
    private readonly string _topicName;

    public ServiceBusPublisher(string connectionString, string topicName)
    {
        _client = new ServiceBusClient(connectionString);
        _topicName = topicName;
    }

    public async Task PublishAsync<T>(T message)
    {
        var sender = _client.CreateSender(_topicName);
        var json = JsonSerializer.Serialize(message);

        var sbMessage = new ServiceBusMessage(json)
        {
            ContentType = "application/json"
        };

        await sender.SendMessageAsync(sbMessage);
    }
}
```

---

3. Order Service

3.1 Order Entity & DbContext

```
public class Order
{
```

```
    public int Id { get; set; }
    public string Status { get; set; } = "Pending";
}
public class OrderDbContext : DbContext
{
    public OrderDbContext(DbContextOptions<OrderDbContext> options) : base(options) { }
    public DbSet<Order> Orders => Set<Order>();
}
```

3.2 Order Controller (Saga Starter)

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderDbContext _db;
    private readonly ServiceBusPublisher _publisher;

    public OrdersController(OrderDbContext db, ServiceBusPublisher publisher)
    {
        _db = db;
        _publisher = publisher;
    }

    [HttpPost]
    public async Task<IActionResult> Create()
    {
```

```
        var order = new Order();
        _db.Orders.Add(order);
        await _db.SaveChangesAsync();

        await _publisher.PublishAsync(new OrderCreatedEvent
        {
            OrderId = order.Id,
            Amount = 5000,
            CorrelationId = Guid.NewGuid()
        });

        return Ok(order);
    }
}
```

3.3 Order Event Listener

```
public class OrderSagaListener : BackgroundService
{
    private readonly ServiceBusProcessor _processor;
    private readonly IServiceScopeFactory _scopeFactory;

    public OrderSagaListener(ServiceBusClient client, IServiceScopeFactory scopeFactory)
    {
        _processor = client.CreateProcessor("order-saga-topic", "order-service-sub");
        _scopeFactory = scopeFactory;
    }
```

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    _processor.ProcessMessageAsync += Handle;
    _processor.ProcessErrorAsync += _ => Task.CompletedTask;
    await _processor.StartProcessingAsync(stoppingToken);
}

private async Task Handle(ProcessMessageEventArgs args)
{
    var json = args.Message.Body.ToString();

    using var scope = _scopeFactory.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<OrderDbContext>();

    if (json.Contains("InventoryReserved"))
    {
        var evt = JsonSerializer.Deserialize<InventoryReservedEvent>(json);
        var order = await db.Orders.FindAsync(evt!.OrderId);
        order!.Status = "Completed";
    }
    else if (json.Contains("PaymentFailed") || json.Contains("InventoryFailed"))
    {
        var evt = JsonSerializer.Deserialize<BaseEvent>(json);
        var orderId = (int)args.Message.ApplicationProperties["OrderId"];
        var order = await db.Orders.FindAsync(orderId);
```

```
            order!.Status = "Cancelled";
        }

        await db.SaveChangesAsync();
        await args.CompleteMessageAsync(args.Message);
    }
}
```

---

4. Payment Service

4.1 Payment Listener

```
public class PaymentSagaListener : BackgroundService
{
    private readonly ServiceBusProcessor _processor;
    private readonly ServiceBusPublisher _publisher;

    public PaymentSagaListener(ServiceBusClient client, ServiceBusPublisher publisher)
    {
        _processor = client.CreateProcessor("order-saga-topic", "payment-service-sub");
        _publisher = publisher;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        _processor.ProcessMessageAsync += Handle;
        _processor.ProcessErrorAsync += _ => Task.CompletedTask;
        await _processor.StartProcessingAsync(stoppingToken);
```

```
}

private async Task Handle(ProcessMessageEventArgs args)
{
    var json = args.Message.Body.ToString();

    if (json.Contains("OrderCreated"))
    {
        var evt = JsonSerializer.Deserialize<OrderCreatedEvent>(json);

        var paymentSuccess = true; // simulate

        if (paymentSuccess)
        {
            await _publisher.PublishAsync(new PaymentCompletedEvent
            {
                OrderId = evt!.OrderId,
                CorrelationId = evt.CorrelationId
            });
        }
        else
        {
            await _publisher.PublishAsync(new PaymentFailedEvent
            {
                OrderId = evt!.OrderId,
                Reason = "Insufficient funds",
```

```
            CorrelationId = evt.CorrelationId
        });
    }
}

if (json.Contains("InventoryFailed"))
{
    var evt = JsonSerializer.Deserialize<InventoryFailedEvent>(json);

    await _publisher.PublishAsync(new PaymentRefundedEvent
    {
        OrderId = evt!.OrderId,
        CorrelationId = evt.CorrelationId
    });
}

await args.CompleteMessageAsync(args.Message);
}
}
```

---

5. Inventory Service

5.1 Inventory Listener

```
public class InventorySagaListener : BackgroundService
{
    private readonly ServiceBusProcessor _processor;
    private readonly ServiceBusPublisher _publisher;
```

```
public InventorySagaListener(ServiceBusClient client, ServiceBusPublisher publisher)
{
    _processor = client.CreateProcessor("order-saga-topic", "inventory-service-sub");
    _publisher = publisher;
}

protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    _processor.ProcessMessageAsync += Handle;
    _processor.ProcessErrorAsync += _ => Task.CompletedTask;
    await _processor.StartProcessingAsync(stoppingToken);
}

private async Task Handle(ProcessMessageEventArgs args)
{
    var json = args.Message.Body.ToString();

    if (json.Contains("PaymentCompleted"))
    {
        var evt = JsonSerializer.Deserialize<PaymentCompletedEvent>(json);

        var stockAvailable = true; // simulate

        if (stockAvailable)
        {
```

```
            await _publisher.PublishAsync(new InventoryReservedEvent
            {
                OrderId = evt!.OrderId,
                CorrelationId = evt.CorrelationId
            });
        }
        else
        {
            await _publisher.PublishAsync(new InventoryFailedEvent
            {
                OrderId = evt!.OrderId,
                Reason = "Out of stock",
                CorrelationId = evt.CorrelationId
            });
        }
    }

    await args.CompleteMessageAsync(args.Message);
  }
}
```

---

6. Program.cs (Same Pattern for All Services)

```
builder.Services.AddSingleton(
  new ServiceBusClient(builder.Configuration["ServiceBus:ConnectionString"]));

builder.Services.AddSingleton(sp =>
```

```
    new ServiceBusPublisher(
        builder.Configuration["ServiceBus:ConnectionString"],
        "order-saga-topic"));

builder.Services.AddHostedService<OrderSagaListener>(); // or Payment/Inventory
```

---

7. How Should Run This

1. Create Azure Service Bus **Topic**
2. Create **3 subscriptions**
3. Run all three services
4. Call POST /api/orders
5. Observe:
    - Events flowing
    - Order completing or cancelling
    - No HTTP calls between services

Final Concept Reinforced

Each service **owns its data**, **reacts to events**, and **publishes the next event**.
No coordinator. No direct calls. Only choreography.
